

AGEX Multi-Tenant Architecture

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX Multi-Tenant Architecture의 구조, Tenant Resource Model, 데이터 격리, 실행 격리, 권한 경계, 설정 상속, Secret, Runtime, Knowledge, Memory, Plugin, Model, Marketplace, Billing, Quota, Observability, Region, Deployment 및 Tenant Lifecycle 기준을 정의한다.

AGEX에서 Multi-Tenancy는 단순히 하나의 Database에 tenant_id 컬럼을 추가하는 구조가 아니다.

AGEX는 하나의 Platform 안에서 다수 Tenant가 Agent, Workflow, Knowledge, Memory, Plugin, Model 및 Application Layer를 독립적으로 구성하고 실행할 수 있어야 하며, 각 Tenant의 데이터, 권한, 비용, Runtime Resource 및 운영 정책은 상호 격리되어야 한다.

본 문서는 다음 설계의 기준으로 사용한다.

- Tenant Resource Model
- Tenant Boundary
- Tenant Context
- Resource Ownership
- Data Isolation
- Runtime Isolation
- Network Isolation
- Secret Isolation
- Identity Isolation
- Knowledge Isolation
- Memory Isolation
- Plugin Installation Isolation
- Model Policy Isolation
- Quota

- Cost
- Billing
- Configuration
- Policy
- Region
- Backup
- Tenant Lifecycle
- Tenant Migration
- Tenant Export
- Tenant Deletion
- Tenant Observability
- Tenant Security

1.2 적용 범위

본 문서는 AGEX Platform 전체에 적용되는 Multi-Tenant 구조를 정의한다.

다음 영역과 직접 연계된다.

- Product Architecture
- Core Architecture
- Runtime Architecture
- Security Architecture
- Agent Framework
- Workflow Engine
- Knowledge Engine
- Memory Engine
- Plugin Framework
- Marketplace

- API Specification
- Deployment Architecture
- Governance
- Operations Guide

1.3 Tenant의 공식 정의

AGEX Tenant는 사용자, Resource, 데이터, 실행, 정책, 설정, Secret, 비용 및 운영 상태를 독립적으로 소유하고 관리하는 최상위 논리적 격리 단위이다.

Tenant는 최소한 다음을 가진다.

- Tenant Identity
- Tenant State
- Tenant Configuration
- Tenant Policy
- Tenant Resource Namespace
- Tenant Users and Roles
- Tenant Secrets
- Tenant Runtime Scope
- Tenant Data Scope
- Tenant Quota
- Tenant Usage
- Tenant Billing Context
- Tenant Audit Context

2. Multi-Tenant Architecture 목표

AGEX Multi-Tenant Architecture는 다음 목표를 충족해야 한다.

2.1 강한 격리

한 Tenant의 사용자, Agent, Workflow, Plugin 또는 Runtime이 다른 Tenant의 Resource 나 데이터에 접근할 수 없어야 한다.

2.2 독립적 운영

Tenant별로 Agent, Workflow, Plugin, Knowledge, Memory 및 Model Policy를 독립적으로 구성할 수 있어야 한다.

2.3 자원 공정성

특정 Tenant가 Platform Resource를 독점하여 다른 Tenant의 서비스 품질을 저하시킬 수 없어야 한다.

2.4 비용 분리

모든 주요 사용량과 비용을 Tenant 기준으로 추적할 수 있어야 한다.

2.5 정책 독립성

Tenant별 Security, Model, Runtime, Retention 및 Approval Policy를 독립적으로 적용할 수 있어야 한다.

2.6 확장 가능한 격리 수준

기본 Shared 구조부터 Dedicated Runtime 또는 Dedicated Infrastructure까지 Tenant별 격리 수준을 선택할 수 있어야 한다.

2.7 데이터 주권

Tenant 정책에 따라 데이터 저장·처리·백업 Region을 제한할 수 있어야 한다.

2.8 Tenant Lifecycle

Tenant 생성부터 Suspended, Export, Migration 및 삭제까지 전체 Lifecycle을 관리할 수 있어야 한다.

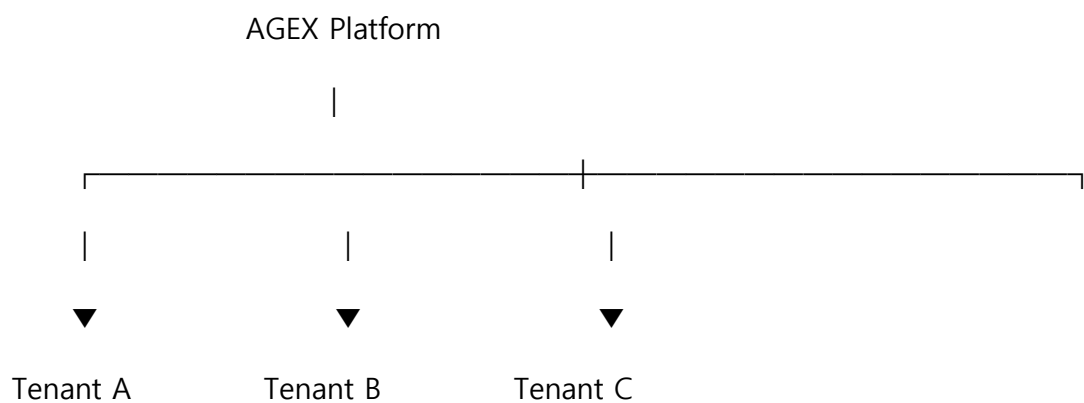
3. Multi-Tenant 핵심 원칙

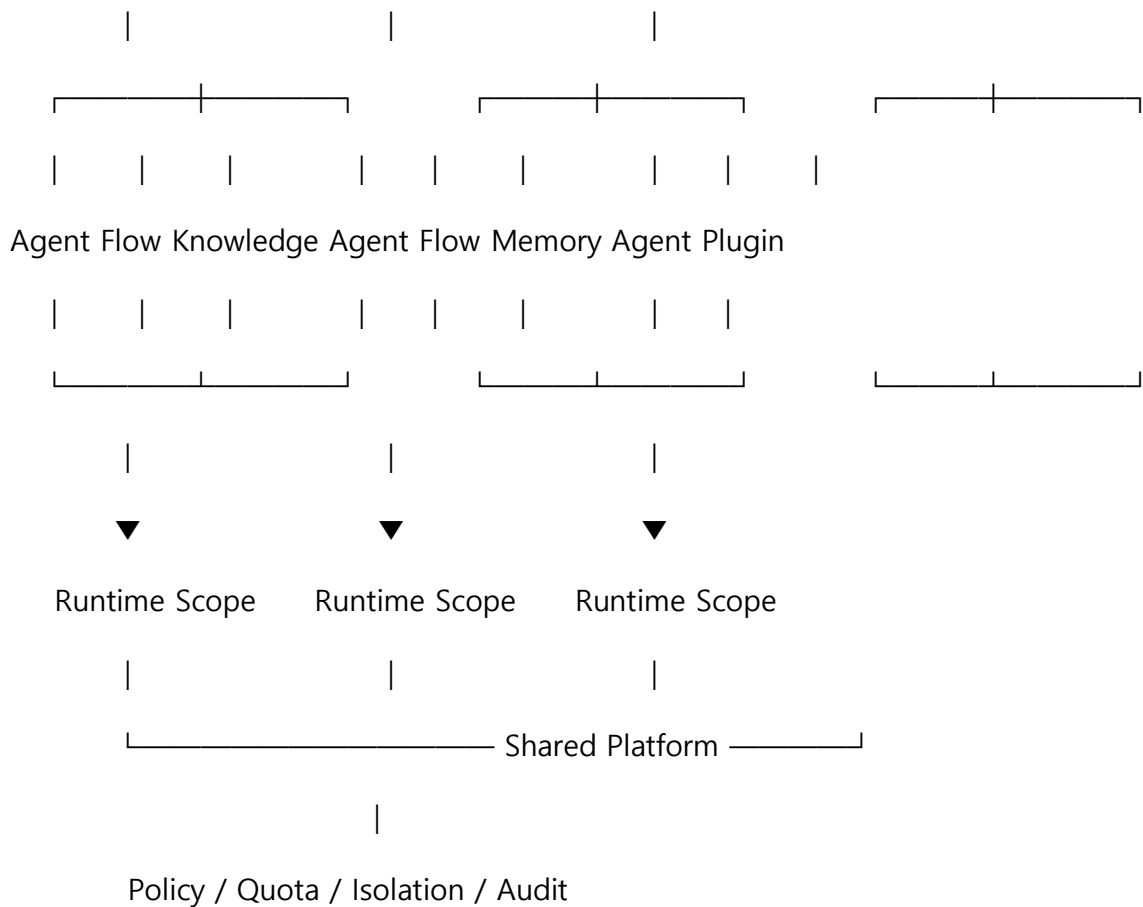
AGEX는 다음 원칙을 따른다.

- Tenant is a First-Class Resource
- Tenant Context Everywhere
- No Tenantless Resource

- Default Tenant Isolation
- Least Privilege
- Resource Ownership by Tenant
- Data Partition by Tenant
- Runtime Fairness
- Configuration Inheritance
- Policy Cannot Weaken Platform Baseline
- Tenant-Specific Secrets
- Tenant-Specific Usage
- Tenant-Specific Audit
- Tenant-Specific Installation
- Region Awareness
- Explicit Cross-Scope Control
- No Hidden Cross-Tenant Sharing
- Deletion Propagation
- Observable by Tenant
- Scalable Isolation

4. Multi-Tenant 전체 구조





Shared Infrastructure를 사용하더라도 Tenant Context는 모든 Layer에서 유지되어야 한다.

5. Tenant Resource Model

5.1 Tenant 공통 속성

Tenant Resource는 최소한 다음 속성을 가진다.

- Tenant ID
- Name
- Display Name
- Status
- Tenant Type
- Region Policy

- Security Profile
- Runtime Profile
- Quota Profile
- Billing Profile
- Data Retention Policy
- Model Policy
- Plugin Policy
- Created At
- Activated At
- Suspended At
- Updated At
- Metadata

5.2 Tenant ID

Tenant ID는 생성 후 변경하지 않는다.

5.3 Tenant Name

사람이 읽는 Tenant Name은 변경 가능할 수 있지만 Tenant ID와 분리한다.

5.4 Tenant 상태

- Provisioning
 - Active
 - Restricted
 - Suspended
 - Closing
 - Deleting
 - Deleted
-

6. Tenant Context

6.1 정의

Tenant Context는 현재 요청과 실행이 어느 Tenant Scope에 속하는지 나타내는 필수 실행 정보이다.

6.2 포함 정보

- Tenant ID
- Tenant Status
- Security Profile
- Policy Reference
- Region
- Quota Context
- Billing Context

6.3 전파 대상

Tenant Context는 다음 모든 영역에 전파되어야 한다.

- API Request
- Internal Service Call
- Execution
- Task
- Queue Message
- Event
- Plugin Request
- Model Request
- Knowledge Retrieval
- Memory Retrieval
- Artifact

- Audit
- Usage

6.4 Tenant Context 누락

Tenant Scope가 필요한 작업에서 Tenant Context를 결정할 수 없으면 실행을 거부한다.

7. Tenant Resolution

7.1 Tenant 결정 Source

Tenant는 다음 정보에서 결정할 수 있다.

- Authentication Token
- Explicit Tenant Header
- Resource Ownership
- Service Context
- Execution Snapshot

7.2 충돌

Token Tenant와 요청 Tenant가 다르면 요청을 거부한다.

7.3 관리자

Platform Administrator가 Tenant를 지정할 경우 별도 Privileged Scope를 요구한다.

7.4 암묵적 Tenant 변경 금지

Session 중 Tenant가 자동으로 변경되지 않도록 한다.

8. Resource Ownership

8.1 기본 원칙

대부분의 AGEX Resource는 하나의 Tenant에 속한다.

8.2 Tenant Resource 예

- Agent

- Workflow
- Knowledge Collection
- Memory
- Plugin Installation
- Model Profile
- Secret
- Artifact
- Policy
- Execution

8.3 Platform Resource

일부 Resource는 Platform Scope에 존재할 수 있다.

예:

- Platform Package
- Global Runtime Type
- Public Marketplace Listing
- Platform Security Baseline

8.4 Platform Resource 사용

Platform Resource를 사용한다고 Tenant 소유권이 공유되는 것은 아니다.

Tenant에는 별도 Installation, Binding 또는 Reference가 생성되어야 한다.

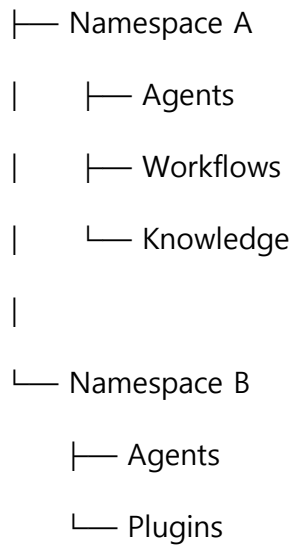
9. Namespace Architecture

9.1 정의

Tenant 내부 Resource를 추가 논리 단위로 구분하기 위해 Namespace를 지원할 수 있다.

구조:

Tenant



9.2 Namespace 목적

- Resource Grouping
- Access Control
- Environment Separation
- Team Separation
- Application Boundary

9.3 Tenant Boundary 우선

Namespace가 Tenant Boundary를 넘을 수 없다.

10. Data Isolation Model

AGEX는 배포 요구 수준에 따라 복수의 데이터 격리 모델을 지원할 수 있다.

10.1 Shared Database / Shared Schema

모든 Tenant가 동일 Schema를 사용하고 Row-level Tenant Key로 분리한다.

10.2 Shared Database / Separate Schema

Tenant별 Database Schema를 분리한다.

10.3 Separate Database

Tenant별 Database를 별도로 운영한다.

10.4 Dedicated Data Plane

고보안 Tenant는 Database, Runtime 및 Network까지 독립적으로 구성할 수 있다.

10.5 선택 기준

다음에 따라 Isolation Level을 결정한다.

- Data Sensitivity
- Tenant Size
- Compliance
- Region
- Performance
- Cost
- Operational Complexity

11. Shared Schema Isolation

Shared Schema 구조를 사용하는 경우 모든 Tenant Resource Table에 Tenant Key를 필수로 적용한다.

예:

tenant_id

resource_id

resource_type

...

11.1 Query Enforcement

Repository Layer에서 Tenant Filter를 강제한다.

11.2 Database-Level Protection

가능한 경우 Row Level Security 또는 이에 준하는 추가 방어를 적용할 수 있다.

11.3 Raw Query 제한

Tenant Filter를 우회할 수 있는 임의 Raw Query 사용을 제한한다.

11.4 Test

모든 Repository는 Cross-Tenant Isolation Test를 가져야 한다.

12. Separate Schema Architecture

Tenant별 Schema 구조는 다음 장점을 가진다.

- 논리적 격리 강화
- Backup 분리
- Migration 통제
- Tenant별 관리 용이

하지만 다음 운영 비용이 증가한다.

- Schema Migration
- Connection Pool
- Backup
- Monitoring

따라서 Tenant 규모와 요구사항에 따라 선택적으로 사용한다.

13. Dedicated Database Architecture

Dedicated Database Tenant는 다음 상황에서 사용할 수 있다.

- 높은 데이터 민감도
- 강한 규제 요구
- 매우 큰 Tenant
- 독립 Backup 요구
- 특정 Region 요구

- 전용 성능 요구

13.1 논리 Contract 동일

저장소가 Dedicated라도 AGEX Domain Contract는 Shared 구조와 동일해야 한다.

Application Code가 Tenant별 저장 구조를 직접 구분하지 않도록 Data Provider 계층에서 추상화한다.

14. Tenant Data Routing

Data Access Layer는 Tenant Context에 따라 저장 위치를 결정한다.

Request

→ Tenant Context

→ Tenant Data Profile

→ Data Routing

→ Appropriate Data Store

14.1 Application 코드

Domain Logic에 Tenant별 Database Endpoint를 Hard Coding하지 않는다.

14.2 Routing Cache

Tenant→Storage Mapping을 Cache할 수 있지만 변경 시 즉시 무효화해야 한다.

15. Identity Isolation

15.1 User와 Tenant 관계

하나의 User Identity가 여러 Tenant에 참여할 수 있다.

그러나 Role과 Permission은 Tenant별로 독립 관리해야 한다.

예:

User X

└─ Tenant A → Administrator

└─ Tenant B → Developer

└─ Tenant C → Viewer

15.2 Global Role 자동 적용 금지

한 Tenant의 관리자 권한이 다른 Tenant로 자동 확장되지 않는다.

15.3 Membership

Tenant Membership은 독립 Resource로 관리할 수 있다.

16. Tenant Membership Resource

공통 속성:

- Membership ID
- Tenant ID
- Principal ID
- Roles
- Status
- Joined At
- Invited By
- Expiration
- Attributes

16.1 상태

- Invited
- Active
- Suspended
- Revoked
- Expired

16.2 Membership 삭제

Membership 삭제가 사용자 Identity 자체 삭제를 의미하지 않는다.

17. Tenant Role

Tenant는 자체 Role을 정의할 수 있다.

단, Platform Security Baseline을 넘는 권한은 부여할 수 없다.

17.1 역할 구성

- Role ID
- Permissions
- Conditions
- Namespace Scope
- Resource Scope

17.2 Platform Reserved Role

일부 역할은 Platform이 예약할 수 있다.

18. Tenant Policy Architecture

18.1 Tenant Policy 대상

- Agent Autonomy
- Model Provider
- Plugin
- Knowledge
- Memory
- Data Classification
- Approval
- Cost
- Retention

- Region
- Runtime

18.2 상속 구조

Platform Security Baseline



Environment Policy



Tenant Policy



Namespace Policy



Resource Policy

18.3 하위 완화 금지

하위 Policy가 상위 최소 기준을 약화시킬 수 없다.

19. Tenant Configuration

19.1 Configuration 범위

- Feature Configuration
- Runtime Defaults
- Model Defaults
- Resource Limits
- UI Preferences
- Event Settings
- Notification Settings

19.2 Secret 제외

Secret 값은 Tenant Configuration에 직접 저장하지 않는다.

19.3 Version

중요 Configuration 변경은 Revision을 기록한다.

19.4 Snapshot

Execution 시작 시 필요한 Tenant Configuration을 Snapshot에 포함할 수 있다.

20. Tenant Secret Isolation

20.1 Secret Scope

모든 Tenant Secret에는 Tenant ID가 있어야 한다.

20.2 Secret 공유

다른 Tenant가 동일한 외부 Provider를 사용하더라도 Credential은 기본적으로 독립 관리한다.

20.3 Platform Managed Secret

Platform이 공동 Credential을 제공하는 경우에도 Tenant에 직접 Secret 값을 노출하지 않는다.

20.4 Encryption Key

필요한 환경에서는 Tenant별 Key로 Secret을 암호화할 수 있다.

21. Agent Multi-Tenancy

21.1 Agent 소유권

Agent Definition은 Platform Package 또는 Tenant Resource일 수 있다.

21.2 Marketplace Agent

Marketplace Agent 설치 시 Tenant 내부에 Installation 또는 Instance Reference를 생성한다.

21.3 Agent 실행

Agent Execution에는 Tenant Context가 반드시 포함된다.

21.4 Agent Memory

Tenant A에서 실행된 Agent Memory를 Tenant B에서 사용할 수 없다.

21.5 Agent 권한

Agent Permission은 Tenant별로 독립 설정한다.

22. Workflow Multi-Tenancy

Workflow는 Tenant Resource로 실행한다.

22.1 공유 Package

동일 Marketplace Workflow Package를 여러 Tenant가 설치할 수 있지만 각 Tenant의 다음 요소는 분리한다.

- Configuration
- Permissions
- Secrets
- Execution
- State
- Cost

22.2 Workflow Instance

Workflow Instance는 정확히 하나의 Tenant에 속해야 한다.

22.3 Child Workflow

Child Workflow도 Parent Tenant를 유지해야 한다.

다른 Tenant로 실행 Context를 이동할 수 없다.

23. Knowledge Multi-Tenancy

23.1 Knowledge Collection

모든 Tenant Knowledge Collection은 Tenant Scope를 가진다.

23.2 Vector Index

Vector Search는 Tenant Filter를 필수로 적용한다.

가능하면 Tenant Partition 또는 Namespace를 사용한다.

23.3 Shared Public Knowledge

Platform Public Knowledge가 존재할 수 있지만 Tenant Private Knowledge와 명확히 분리해야 한다.

구조:

Knowledge Retrieval

└─ Platform Public Scope

└─ Current Tenant Scope

23.4 자동 공유 금지

Tenant Knowledge를 다른 Tenant의 Retrieval 대상으로 자동 사용하지 않는다.

24. Memory Multi-Tenancy

24.1 모든 Memory에 Tenant ID 적용

User Memory 역시 Tenant Context를 가져야 할 수 있다.

24.2 User Across Tenants

같은 사용자가 여러 Tenant에 속해도 Tenant별 Memory를 기본 분리한다.

24.3 Global User Memory

Platform 전체 범위 User Memory가 필요한 경우 Tenant Memory와 별도의 명시적 Scope로 정의해야 한다.

24.4 기본값

Tenant Context가 있는 업무 실행에서는 Tenant Memory가 우선이며 다른 Tenant Memory를 사용할 수 없다.

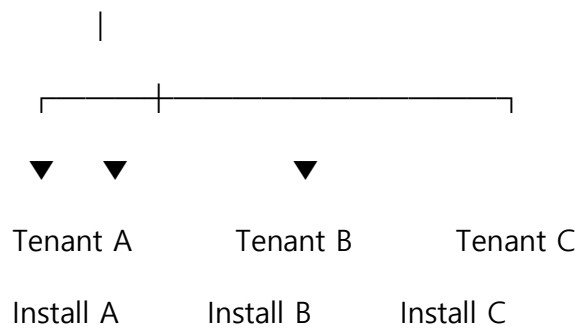
25. Plugin Multi-Tenancy

25.1 Plugin Definition과 Installation

Plugin Definition은 공유될 수 있다.

Plugin Installation은 Tenant별로 독립된다.

Plugin Definition v1.3



25.2 Installation별 분리

- Credential
- Config
- Permission
- Network
- Quota
- Status
- Usage

25.3 Runtime Context

Plugin Action 실행 시 하나의 Tenant Installation만 포함한다.

26. Marketplace Multi-Tenancy

26.1 Public Marketplace

여러 Tenant가 공통 Catalog를 탐색할 수 있다.

26.2 Installation

설치는 항상 특정 Tenant에 귀속된다.

26.3 Private Marketplace

Tenant 전용 Package Catalog를 제공할 수 있다.

26.4 Private Package

Tenant Private Package는 Public Marketplace에 자동 공개되지 않는다.

26.5 Entitlement

Package License와 Subscription도 Tenant 기준으로 관리한다.

27. Model Multi-Tenancy

27.1 Model Registry

Model Definition은 Platform Scope에서 공유할 수 있다.

27.2 Tenant Model Profile

Tenant별로 다음을 설정한다.

- Allowed Models
- Allowed Providers
- Preferred Provider
- Cost Limit
- Data Policy
- Region
- Fallback

27.3 Provider Credential

Tenant가 자체 Provider Credential을 사용할 수 있다.

27.4 Shared Provider

Platform 공동 Provider를 사용하는 경우 Usage와 Cost는 Tenant별로 분리 측정한다.

28. Model Routing Isolation

Model Router는 Tenant Policy를 반드시 적용한다.

Model Request

- Tenant
- Tenant Model Policy
- Data Classification
- Allowed Providers
- Budget
- Route

28.1 다른 Tenant 설정 참조 금지

Provider 선택에서 다른 Tenant의 Model Preference를 사용하지 않는다.

28.2 Shared Cache 주의

Model Response Cache를 사용하는 경우 Tenant와 Data Classification을 Cache Key에 포함해야 한다.

29. Runtime Multi-Tenancy

AGEX Runtime은 Shared 또는 Dedicated Worker Pool을 지원할 수 있다.

29.1 Shared Pool

다수 Tenant Task를 공통 Worker Pool에서 처리한다.

29.2 Isolated Pool

보안 또는 성능 요구가 높은 Tenant는 별도 Pool을 사용한다.

29.3 Dedicated Runtime

특정 Tenant 전용 Runtime을 구성할 수 있다.

29.4 동일 Contract

어떤 Deployment Model이든 Execution Contract는 동일해야 한다.

30. Runtime Tenant Context

Task Dispatch 시 최소한 다음 정보를 전달한다.

- Tenant ID
- Execution ID
- Security Context
- Resource Limits
- Data Policy
- Runtime Profile

30.1 Worker 초기화

Worker는 Task 처리 전에 Tenant Context를 확인해야 한다.

30.2 Worker 재사용

Shared Worker가 다음 Tenant Task를 처리할 때 이전 Tenant 상태가 남지 않아야 한다.

30.3 Local Cache

Tenant별 데이터가 Worker Local Cache에 남는 경우 안전한 격리와 제거가 필요하다.

31. Noisy Neighbor Protection

특정 Tenant 부하가 다른 Tenant에 영향을 주는 것을 방지한다.

31.1 제어 대상

- API Requests
- Concurrent Executions
- Model Calls
- Plugin Calls
- CPU
- Memory
- GPU

- Queue
- Storage
- Knowledge Retrieval

31.2 제어 방식

- Rate Limit
- Quota
- Concurrency Limit
- Weighted Fair Queue
- Resource Reservation
- Priority Aging
- Worker Pool Isolation

32. Tenant Quota Architecture

Tenant는 다음 Quota를 가질 수 있다.

- API Requests
- Agent Executions
- Workflow Executions
- Concurrent Tasks
- Model Tokens
- Plugin Calls
- Knowledge Storage
- Memory Storage
- Artifact Storage
- Compute
- GPU

- Network

32.1 Hard Quota

초과 시 실행을 거부한다.

32.2 Soft Quota

경고 후 계속 사용할 수 있다.

32.3 Budget Quota

비용 금액을 기준으로 제한할 수 있다.

32.4 Quota Override

관리자 Override는 Audit 대상이다.

33. Tenant Fair Scheduling

Runtime Scheduler는 Tenant 간 공정성을 제공해야 한다.

33.1 Weighted Fairness

Tenant Plan 또는 정책에 따라 Weight를 부여할 수 있다.

33.2 Critical Task

Priority가 높아도 Tenant 전체 자원을 무제한 사용할 수 없다.

33.3 Starvation 방지

낮은 Priority Tenant가 영구 대기하지 않도록 Priority Aging 등을 사용할 수 있다.

34. Tenant Cost Architecture

모든 주요 Usage는 Tenant ID를 기준으로 집계한다.

34.1 비용 항목

- Model
- Token
- Plugin

- Compute
- GPU
- Storage
- Network
- Knowledge
- Memory
- Marketplace Package

34.2 하위 분석

비용은 다음 단위로 추가 분해할 수 있다.

Tenant

└─ Application

└─ Agent

└─ Workflow

└─ Model

└─ Plugin

└─ User

34.3 비용 정확성

Retry, 중복 Event 및 실패 실행으로 이중 집계되지 않도록 Idempotency를 적용한다.

35. Billing Context

Tenant는 다음 Billing 정보를 가질 수 있다.

- Billing Account
- Plan
- Entitlements
- Credit

- Budget
- Billing Status

35.1 Billing과 Authorization

결제 상태가 정상이라고 Security Permission이 자동 부여되지는 않는다.

35.2 Entitlement

Plan에 따라 사용 가능한 Feature를 제한할 수 있다.

36. Tenant Entitlement

Entitlement는 다음을 정의할 수 있다.

- Feature
- Maximum Resource
- Runtime Class
- Marketplace Access
- Model Access
- Support Level

36.1 Entitlement와 Permission 구분

Entitlement는 제품 사용 권리이다.

Permission은 특정 Principal의 행동 권한이다.

둘은 별도로 평가해야 한다.

37. Tenant Event Isolation

모든 Tenant Event는 Tenant Context를 가진다.

37.1 Event Bus

Shared Event Bus에서도 Tenant Scope를 유지해야 한다.

37.2 Subscription

Tenant Consumer는 자신의 Tenant Event만 구독할 수 있다.

37.3 Platform Event

Platform Scope Event는 별도 Namespace로 관리한다.

37.4 DLQ

Dead Letter Queue에서도 Tenant Metadata를 유지한다.

38. Tenant Queue Isolation

Queue 전략은 다음 중 하나를 사용할 수 있다.

- Shared Queue + Tenant Metadata
- Partitioned Queue
- Tenant Class Queue
- Dedicated Tenant Queue

38.1 선택 기준

- 부하
- 보안
- SLA
- 운영 비용
- Tenant 규모

38.2 Queue가 권한 경계가 아님

Queue가 분리되어 있어도 Runtime에서 Tenant Context를 다시 검증한다.

39. Tenant Cache Isolation

39.1 Cache Key

Tenant Resource Cache Key에는 Tenant ID를 포함한다.

예:

tenant:{tenant_id};agent:{agent_id}

39.2 Public Resource

Platform Public Resource는 별도 Cache Namespace를 사용할 수 있다.

39.3 Permission-aware Cache

Permission에 따라 결과가 달라지는 데이터는 Principal 또는 Policy Context도 고려해야 한다.

40. Tenant Search Isolation

Search Index를 공유하더라도 Tenant Filter를 검색 단계에서 강제한다.

40.1 Post-Filter만 의존 금지

검색 후 Application 코드에서 Tenant 데이터를 제거하는 방식만으로 격리를 보장하지 않는다.

40.2 Search Document

모든 Indexed Document에 Tenant Scope를 기록한다.

41. Artifact Multi-Tenancy

Artifact는 다음 정보를 가져야 한다.

- Artifact ID
- Tenant ID
- Owner
- Classification
- Retention
- Encryption
- Access Policy

41.1 저장 경로

공유 Object Storage를 사용하는 경우 Tenant Prefix 또는 Namespace를 분리한다.

41.2 Signed URL

Tenant Context를 검증한 후에만 생성한다.

41.3 다른 Tenant URL 재사용

Tenant A의 Signed URL이 Tenant B에서 권한 확대 수단으로 사용되지 않도록 한다.

42. Tenant Encryption Architecture

42.1 기본

모든 Tenant 데이터는 저장 시 암호화한다.

42.2 Key Isolation 수준

- Platform Shared Key
- Tenant-Derived Key
- Tenant Dedicated Key
- External Customer Managed Key

42.3 요구사항

배포 모델에 따라 선택할 수 있어야 한다.

42.4 Key Rotation

Tenant 서비스 중단을 최소화하며 Rotation할 수 있어야 한다.

43. Customer-Managed Keys

고급 배포에서는 Tenant가 자체 Key를 관리하는 구조를 지원할 수 있다.

43.1 CMK 상태

Key가 폐기되거나 접근 불가능해지면 해당 Tenant 데이터 접근이 중단될 수 있다.

43.2 Platform 책임

Key Material을 임의로 복제하거나 보관하지 않는다.

43.3 Rotation

Tenant와 Platform 간 명시적 Key Version 관리가 필요하다.

44. Tenant Network Isolation

44.1 Shared Network

기본 SaaS는 공유 Network에서 논리적 정책으로 분리할 수 있다.

44.2 Dedicated Network

높은 보안 수준 Tenant는 다음을 사용할 수 있다.

- Dedicated Network Segment
- Private Endpoint
- Dedicated Egress
- Private Runtime Pool

44.3 Tenant Egress Policy

Tenant별 외부 연결 허용 목록을 설정할 수 있다.

45. Region Architecture

Tenant는 하나 이상의 허용 Region을 가질 수 있다.

45.1 Region Policy

- Primary Region
- Allowed Processing Regions
- Backup Region
- Prohibited Regions

45.2 적용 대상

- Data Store
- Object Store
- Runtime

- Knowledge Index
- Memory
- Model Provider
- Backup

45.3 정책 우선

가용성 문제로 인해 금지 Region으로 자동 이동하지 않는다.

46. Tenant Placement

Tenant Provisioning 시 배치 정보를 결정한다.

예:

Tenant

→ Region

→ Data Cluster

→ Runtime Pool

→ Storage

→ Encryption Profile

46.1 Placement 변경

운영 중 Placement 변경은 Tenant Migration으로 처리한다.

47. Tenant Provisioning

표준 생성 흐름:

Tenant Request

→ Validate

→ Allocate Tenant ID

→ Select Region

- Assign Data Profile
- Create Security Context
- Create Default Policy
- Configure Quota
- Create Audit Context
- Activate

47.1 초기 상태

Provisioning 중에는 일반 실행을 허용하지 않는다.

47.2 실패

일부 Resource Provisioning에 실패하면 반쯤 생성된 Active Tenant로 남기지 않는다.

48. Tenant Default Resources

필요한 경우 Tenant 생성 시 기본 Resource를 생성할 수 있다.

예:

- Default Namespace
- Default Policy
- Default Model Profile
- Default Quota
- Default Role

48.1 최소화

특정 업무 Workflow나 Agent를 기본 생성하지 않는다.

AGEX는 특정 Application 구조를 Tenant에 강제하지 않는다.

49. Tenant Lifecycle

Tenant Lifecycle은 다음과 같이 정의한다.

Provisioning

→ Active

→ Restricted

→ Suspended

→ Closing

→ Deleting

→ Deleted

49.1 Active

정상 사용 가능 상태이다.

49.2 Restricted

일부 기능이 제한된 상태이다.

49.3 Suspended

신규 실행과 일반 접근을 차단한다.

49.4 Closing

Tenant 종료 준비 상태이다.

49.5 Deleting

데이터 삭제 작업이 진행 중이다.

50. Tenant Suspension

Tenant가 Suspended 상태가 되는 이유는 다음과 같을 수 있다.

- Security Incident
- Administrative Action
- Contract Status
- Billing Policy
- Tenant Request

50.1 신규 실행

기본적으로 차단한다.

50.2 진행 중 실행

정책에 따라 다음을 선택한다.

- Pause
- Cancel
- Terminate
- Finish Safe Tasks

50.3 관리자 접근

복구 목적의 제한된 관리자 접근만 허용할 수 있다.

51. Tenant Reactivation

재활성화 전에 다음을 검증한다.

- Suspension Reason 해소
- Credential 상태
- Policy 상태
- Data Store Health
- Quota
- Runtime Compatibility
- Security Incident 상태

Reactivation은 Audit에 기록한다.

52. Tenant Closing

Tenant가 서비스 종료를 요청하면 즉시 데이터 삭제부터 수행하지 않는다.

표준 흐름:

Close Request

- Stop New Provisioning
 - Stop New Executions
 - Resolve Active Executions
 - Export Option
 - Retention Review
 - Delete Schedule
-

53. Tenant Export

Tenant는 정책과 권한에 따라 자신의 Resource와 데이터를 Export할 수 있어야 한다.

53.1 Export 대상

- Agents
- Workflows
- Configurations
- Knowledge
- Memory
- Plugin Installation Metadata
- Audit
- Usage
- Artifacts

53.2 제외 대상

다음은 직접 Export되지 않을 수 있다.

- Platform Internal Secrets
- Other Tenant Data
- Provider Master Credentials

- Platform Internal Security Metadata

53.3 비동기 처리

대규모 Export는 비동기 Operation으로 수행한다.

54. Tenant Portability

Tenant Export Format은 가능한 한 다음을 유지해야 한다.

- Resource ID
- Version
- Metadata
- Relationships
- Schema Version
- Source Reference

54.1 Provider-specific 데이터

직접 이식이 불가능한 Provider-specific 상태는 명시적으로 표시한다.

54.2 Secret

Secret 원문을 기본 Export에 포함하지 않는다.

55. Tenant Migration

Tenant를 다른 Region, Cluster 또는 Isolation Profile로 이전할 수 있어야 한다.

55.1 Migration 대상

- Database
- Object Storage
- Knowledge Index
- Memory Index
- Runtime Placement

- Encryption Key

55.2 Migration 단계

Plan

- Compatibility Check
- Snapshot
- Replicate
- Validate
- Quiesce Writes
- Final Sync
- Switch Routing
- Verify
- Cleanup

55.3 Downtime

가능한 한 최소화한다.

55.4 Rollback

Cutover 전에 Rollback Point를 유지해야 한다.

56. Tenant Deletion

Tenant 삭제는 단일 DELETE Query가 아니다.

56.1 삭제 대상

- Resources
- Executions
- Knowledge
- Memory
- Artifacts

- Plugin Configuration
- Secrets
- Search Index
- Vector Index
- Cache
- Read Models

56.2 Audit

삭제 요청과 완료 사실에 대한 최소 Audit Metadata는 별도 정책에 따라 유지할 수 있다.

56.3 Backup

Backup Retention과 실제 삭제 시점을 관리해야 한다.

56.4 Tombstone

삭제된 Tenant ID의 재사용을 금지하는 Tombstone을 유지할 수 있다.

57. Tenant Restore

일반 삭제 완료 후 자동 Restore를 보장하지 않는다.

57.1 Grace Period

Tenant Closing 단계에서 제한적인 복구 가능 기간을 둘 수 있다.

57.2 Deleted 상태

최종 삭제 후에는 새로운 Tenant로 재생성해야 할 수 있다.

58. Tenant Backup Architecture

58.1 Backup Scope

배포 모델에 따라 다음을 백업한다.

- Database

- Object Storage
- Resource Definition
- Knowledge Source Metadata
- Memory
- Configuration
- Audit

58.2 Tenant 단위 복구

가능한 Isolation Model에서는 개별 Tenant 단위 Restore를 지원하는 것을 목표로 한다.

58.3 Shared Database

Shared 환경에서는 Tenant 부분 Restore가 복잡하므로 별도 Logical Export 또는 Point-in-Time Recovery 절차가 필요하다.

59. Tenant Disaster Recovery

Tenant의 Disaster Recovery Profile은 다음을 정의할 수 있다.

- Recovery Region
- RPO Class
- RTO Class
- Backup Frequency
- Runtime Recovery Class

정확한 수치는 Deployment 및 Service Level 문서에서 정의한다.

60. Tenant Observability

Tenant는 자신의 범위 내에서 다음을 관측할 수 있어야 한다.

- API Usage
- Agent Executions

- Workflow Executions
- Model Usage
- Plugin Usage
- Knowledge Usage
- Memory Usage
- Errors
- Cost
- Quota
- Security Events

60.1 Platform Observability

Platform Operator는 전체 시스템 상태를 볼 수 있지만 Tenant Content는 최소화해 노출한다.

61. Tenant Metrics Isolation

Metrics Label에 Tenant ID를 사용할 수 있다.

단, 지나치게 높은 Cardinality가 Platform 성능에 영향을 주지 않도록 설계한다.

61.1 외부 모니터링

Tenant가 자체 Monitoring으로 Metric을 Export할 수 있는 기능을 제공할 수 있다.

62. Tenant Log Isolation

Tenant 운영자가 자신의 Log를 볼 수 있는 경우 Tenant Scope를 강제한다.

62.1 Shared Log Store

검색 Query 자체에 Tenant Filter를 강제한다.

62.2 Platform Log

Tenant에게 내부 Platform Debug Log 전체를 노출하지 않는다.

63. Tenant Audit Isolation

각 Audit Record는 Tenant Context를 가져야 한다.

63.1 Tenant Auditor

Tenant Auditor는 자신의 Tenant Audit만 조회한다.

63.2 Platform Audit

Platform Scope 행위는 별도 Audit Scope로 관리한다.

63.3 Cross-Tenant 관리자 행위

Platform 관리자가 Tenant Resource에 접근한 경우 대상 Tenant를 명확하게 기록한다.

64. Tenant Security Profile

Tenant별 Security Profile은 다음을 포함할 수 있다.

- MFA Requirement
- Session Policy
- Data Classification Policy
- Encryption Policy
- Model Provider Policy
- Plugin Trust Requirement
- Runtime Isolation Level
- Audit Level
- Network Policy
- Export Policy

64.1 Baseline

Security Profile은 Platform Baseline보다 약하게 설정할 수 없다.

65. Tenant Runtime Profile

Runtime Profile은 다음을 정의할 수 있다.

- Shared / Isolated / Dedicated
- Worker Class
- Maximum Concurrency
- Sandbox Requirement
- Region
- GPU Access
- Network Egress
- Timeout Limits

65.1 Resource 자동 확대

Tenant가 자신에게 허용된 Entitlement 이상 Runtime Profile을 설정할 수 없다.

66. Tenant Model Profile

Tenant Model Profile은 다음을 포함한다.

- Allowed Providers
- Allowed Models
- Preferred Model
- Fallback
- Data Classification Rules
- Region
- Maximum Cost
- Maximum Token
- External Provider Policy

66.1 공통 Platform Model

Platform 기본 Model이 있어도 Tenant 정책을 우선 검토한다.

67. Tenant Plugin Policy

Tenant별로 다음을 정의할 수 있다.

- Allowed Trust Levels
- Allowed Publishers
- Allowed Permissions
- External Network Rules
- Installation Approval
- Auto Upgrade Policy

67.1 Community Plugin

특정 Tenant는 Community Plugin 설치를 완전히 금지할 수 있다.

68. Tenant Marketplace Policy

Tenant 관리자는 다음을 통제할 수 있다.

- Public Marketplace Access
 - Private Marketplace
 - Paid Package
 - Package Approval
 - Publisher Allowlist
 - Package Blocklist
 - Maximum Spend
-

69. Tenant Knowledge Policy

Tenant는 다음을 설정할 수 있다.

- Allowed Knowledge Sources
 - External Connector
 - External Embedding Provider
 - Retention
 - Data Classification
 - Sharing Scope
 - Index Region
-

70. Tenant Memory Policy

Tenant는 다음을 설정할 수 있다.

- User Memory Enabled
- Agent Memory Enabled
- Maximum Retention
- Sensitive Memory
- External Model Processing
- Memory Export
- Memory Deletion

70.1 사용자 옵션

Tenant Policy가 허용하는 범위 안에서 사용자 단위 Memory Opt-out을 지원할 수 있다.

71. Tenant Resource Quotas

Resource 생성 수 자체에도 제한을 둘 수 있다.

예:

- Maximum Agents
- Maximum Workflows

- Maximum Plugin Installations
- Maximum Knowledge Collections
- Maximum Memory Storage
- Maximum Secrets

71.1 Soft Limit

관리 목적의 Warning으로 사용할 수 있다.

71.2 Hard Limit

신규 Resource 생성을 차단한다.

72. Tenant Rate Limiting

Rate Limit은 다음 조합으로 적용할 수 있다.

Platform Limit

$\cap$

Tenant Limit

$\cap$

Principal Limit

$\cap$

Endpoint Limit

72.1 Tenant Burst

일시적인 Burst Capacity를 허용할 수 있다.

72.2 Abuse

Tenant가 분산 Credential을 이용해 한도를 우회하지 못하도록 집계한다.

73. Tenant Concurrency

동시 실행 제한은 다음 수준으로 적용할 수 있다.

- Tenant
- Agent
- Workflow
- Plugin
- Model

73.1 Parent/Child

Child Execution도 Tenant 전체 Concurrency에 포함한다.

74. Tenant Cost Budget

Tenant는 다음 Budget을 가질 수 있다.

- Daily
- Monthly
- Execution
- Model
- Plugin
- Marketplace

74.1 Budget Threshold

예:

- 50% Notification
- 80% Warning
- 100% Hard Limit

구체 값은 Tenant 설정으로 정의한다.

74.2 Security

Budget 초과를 Agent가 무시할 수 없다.

75. Tenant Service Tier

향후 제품 운영에서 Tenant별 Service Tier를 제공할 수 있다.

예:

- Shared
- Enhanced
- Dedicated

이는 Business Model에서 최종 정의하며 Architecture는 다음 차이를 지원할 수 있어야 한다.

- Quota
- Runtime Isolation
- Support
- Region
- SLA
- Dedicated Resources

76. Shared Tenant Architecture

Shared 구조는 비용 효율성과 확장성을 우선한다.

76.1 공유 가능한 영역

- API Gateway
- Core Services
- Event Infrastructure
- Shared Runtime Pool
- Shared Database Cluster
- Shared Object Storage

76.2 공유 불가능한 논리 영역

- Tenant Resource Scope
- Permissions
- Secrets
- Usage
- Audit Context
- Memory
- Private Knowledge

77. Dedicated Tenant Architecture

Dedicated Tenant는 다음 Resource를 독립적으로 가질 수 있다.

- Database
- Runtime
- Network
- Storage
- Encryption Key
- Model Endpoint

77.1 Control Plane

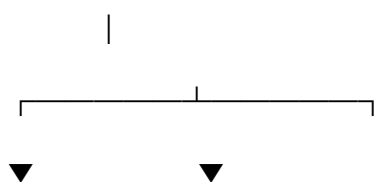
Platform 공통 Control Plane을 사용할 수 있다.

77.2 Data Plane

Tenant 전용 Data Plane을 제공할 수 있다.

구조:

AGEX Shared Control Plane



Shared Data Dedicated Data Plane

Plane Tenant X

78. Control Plane Multi-Tenancy

Control Plane은 다음을 공통 관리할 수 있다.

- Tenant Registry
- Resource Metadata
- Policy
- Deployment
- Marketplace
- Billing

78.1 Tenant Data 최소화

Control Plane에 실제 Tenant Business Data 전체를 복제하지 않는다.

79. Data Plane Multi-Tenancy

Data Plane은 실제 실행과 데이터 처리 영역이다.

- Runtime
- Model Invocation
- Plugin Execution
- Knowledge Retrieval
- Memory Retrieval
- Artifact Processing

79.1 격리 수준

Tenant Profile에 따라 Shared 또는 Dedicated가 가능해야 한다.

80. Cross-Tenant Collaboration

AGEX Core의 기본 Tenant Resource Model은 Cross-Tenant 데이터 공유를 허용하지 않는다.

향후 협업 기능이 필요하다면 일반 Permission 확대가 아니라 별도의 명시적 Federation/Sharing Resource로 설계해야 한다.

80.1 금지

Tenant A Resource ID를 Tenant B Permission에 직접 추가하는 방식으로 공유하지 않는다.

80.2 이유

- Ownership 불명확
 - Revocation 복잡
 - Audit 혼란
 - Data Residency 문제
 - Lifecycle 문제
-

81. Platform Public Resource

Platform Public Resource는 여러 Tenant가 참조할 수 있다.

예:

- Certified Plugin Definition
- Public Agent Package
- Public Knowledge Package
- Model Definition

81.1 읽기 전용 Reference

Tenant가 Platform Public Definition 자체를 수정할 수 없다.

81.2 Tenant Customization

필요하면 Tenant 내부에 별도 Configuration 또는 Derived Resource를 생성한다.

82. Tenant Customization

Tenant는 Public Package를 자신의 환경에 맞게 구성할 수 있다.

단, 다음은 원본 Package와 분리한다.

- Configuration
- Policy
- Credential
- Permission
- Runtime Limit

82.1 Fork

Package Definition 자체를 수정하려면 별도 Tenant-owned Fork Resource로 생성할 수 있다.

83. Tenant Data Import

Tenant는 외부 데이터를 Import할 수 있다.

83.1 Import 검증

- Tenant Ownership
- Schema
- Malware
- Classification
- Size
- Resource Conflict
- Version

83.2 Cross-Tenant Import

다른 Tenant Export를 Import하는 경우에도 별도의 권한 및 소유권 검증이 필요하다.

84. Tenant Resource ID Collision

Resource ID는 Platform 전체에서 고유하게 생성하는 것을 권장한다.

84.1 이유

- Event Correlation
- Audit
- Migration
- Distributed System
- Marketplace

84.2 권한

Global ID가 고유하더라도 ID만으로 접근 권한이 생기지 않는다.

85. Tenant Schema Evolution

Database Schema가 변경되어도 Tenant Isolation이 유지되어야 한다.

85.1 Shared Schema

Migration은 모든 Tenant에 영향을 줄 수 있으므로 호환성 검증이 필요하다.

85.2 Separate Schema

Tenant별 Migration Progress를 관리할 수 있어야 한다.

85.3 Version Drift

너무 많은 Tenant별 Schema Version Drift를 허용하지 않는 것을 원칙으로 한다.

86. Tenant Deployment Version

Platform은 Tenant별 Feature Rollout을 지원할 수 있다.

86.1 Feature Flag

Tenant별로 신규 기능을 단계적으로 활성화할 수 있다.

86.2 Core Contract

Feature Flag가 Resource Contract를 무작위로 변경하지 않도록 한다.

86.3 보안 Patch

Critical Security Patch는 Feature Opt-out 대상이 아닐 수 있다.

87. Tenant Maintenance

Tenant 단위 Maintenance Mode를 지원할 수 있다.

87.1 적용

- Migration
- Security Recovery
- Large Import
- Data Repair

87.2 상태

Maintenance 중 신규 실행을 제한할 수 있다.

87.3 사용자 표시

Tenant 사용자는 상태를 확인할 수 있어야 한다.

88. Tenant Administrative Operations

다음 행위는 강화된 Audit 대상이다.

- Suspend Tenant
- Delete Tenant
- Change Region
- Change Runtime Profile
- Change Encryption Profile
- Change Quota

- Bulk Export
- Security Override

88.1 Reason

고위험 Action은 Reason을 필수로 할 수 있다.

89. Tenant Support Access

운영 지원 목적으로 Platform Staff가 Tenant 환경에 접근해야 할 수 있다.

89.1 원칙

기본적으로 접근하지 않는다.

89.2 지원 접근

필요 시 다음을 적용한다.

- Explicit Permission
- Limited Scope
- Time Limit
- Reason
- Audit
- Tenant Notification Policy

89.3 Content 최소화

지원 담당자에게 필요한 Metadata만 우선 제공한다.

90. Tenant Impersonation 금지

지원 편의를 위해 운영자가 사용자의 Session을 그대로 가장하는 구조를 기본으로 제공하지 않는다.

필요한 경우 명시적 Support Session Resource와 Audit를 사용한다.

91. Tenant Security Incident Isolation

특정 Tenant에서 보안 사고가 발생한 경우 해당 Tenant만 격리할 수 있어야 한다.

91.1 조치

- Suspend Tenant
- Revoke Credentials
- Stop Runtime
- Block Plugins
- Disable External Egress
- Rotate Keys

91.2 Platform 사고

Tenant Isolation Failure 가능성이 있다면 전체 Platform Incident로 승격한다.

92. Tenant Usage Metering

모든 Usage Record는 최소한 다음을 가진다.

- Tenant ID
- Resource Type
- Resource ID
- Execution ID
- Usage Type
- Quantity
- Unit
- Timestamp

92.1 Aggregation

시간 단위, 일 단위, 월 단위로 집계할 수 있다.

92.2 Raw Usage

정산에 사용되는 Usage Record는 변경 추적이 가능해야 한다.

93. Tenant Billing Isolation

한 Tenant의 결제 상태, Credit, Usage 및 Invoice가 다른 Tenant에 노출되어서는 안 된다.

93.1 Marketplace

Publisher에게도 Tenant의 전체 Billing 정보를 노출하지 않는다.

93.2 Usage Sharing

Publisher Settlement에 필요한 최소 Usage만 제공한다.

94. Tenant API Isolation

Public API는 Tenant Context를 강제한다.

예:

GET /agents/{id}

이 요청은 Agent ID가 존재하더라도 현재 Tenant에 속하지 않으면 반환하지 않는다.

94.1 Enumeration 방지

권한이 없는 Resource 존재 여부를 과도하게 노출하지 않는다.

95. Tenant Eventual Consistency

Multi-Tenant Read Model과 Search Index는 Eventual Consistency를 사용할 수 있다.

단, 다음 Security State는 신속하거나 강한 일관성을 요구한다.

- Tenant Suspension
- Permission Revocation
- Secret Revocation
- Policy Block

- Package Block

95.1 Security 우선

Read Model이 오래되어도 Security Decision은 Source of Truth를 우선해야 한다.

96. Tenant Failure Isolation

한 Tenant의 다음 문제는 다른 Tenant로 전파되지 않아야 한다.

- Workflow Failure
- Plugin Failure
- Queue Flood
- Model Abuse
- Memory Explosion
- Knowledge Reindex
- Cost Spike

96.1 Circuit Breaker

Tenant별 Circuit Breaker 또는 Throttling을 적용할 수 있다.

97. Tenant Scalability

Architecture는 다음 규모 증가에 대응해야 한다.

- Tenant Count
- Resource Count per Tenant
- User Count per Tenant
- Execution Throughput
- Storage
- Knowledge Index
- Memory Index

97.1 Tenant Sharding

필요하면 Tenant ID를 기반으로 Data Store를 Shard할 수 있다.

97.2 Large Tenant

대규모 Tenant는 Dedicated Shard 또는 Cluster로 이동할 수 있어야 한다.

98. Tenant Placement Rebalancing

특정 Cluster의 부하가 증가하면 Tenant를 다른 Cluster로 이동할 수 있다.

98.1 자동 이동 제한

Data Residency 또는 Dedicated 정책을 위반하는 이동은 허용하지 않는다.

98.2 Migration Audit

Tenant 이동 이력을 기록한다.

99. Tenant Testing

Multi-Tenant Architecture는 최소 다음 테스트를 수행해야 한다.

99.1 Resource Isolation Test

Tenant A의 Resource를 Tenant B가 조회·수정·삭제할 수 없어야 한다.

99.2 Execution Isolation Test

다른 Tenant Execution을 제어할 수 없어야 한다.

99.3 Knowledge Isolation Test

Vector/Search 결과에 다른 Tenant 데이터가 포함되지 않아야 한다.

99.4 Memory Isolation Test

User 및 Agent Memory가 다른 Tenant로 유출되지 않아야 한다.

99.5 Plugin Isolation Test

Credential과 Configuration이 분리되어야 한다.

99.6 Secret Isolation Test

Cross-Tenant Secret Reference를 거부해야 한다.

99.7 Queue Isolation Test

Task가 잘못된 Tenant Context로 실행되지 않아야 한다.

99.8 Cache Isolation Test

Tenant Cache Key 충돌이 없어야 한다.

99.9 Quota Test

한 Tenant의 자원 초과가 다른 Tenant에 영향을 주지 않아야 한다.

99.10 Migration Test

Tenant 이동 후 Resource와 Permission이 유지되어야 한다.

100. Tenant Chaos Test

장애 상황에서도 격리를 검증한다.

예:

- Tenant DB 장애
- Tenant Runtime Flood
- Queue Redelivery
- Cache Corruption
- Search Index 장애
- Model Provider 장애

특정 Tenant 장애가 Platform 전체 장애로 확대되는지 확인한다.

101. Tenant Performance Test

다음 항목을 검증한다.

- Tenant 수 증가
- Large Tenant

- Small Tenant 다수
 - Concurrent Execution
 - Queue Fairness
 - Search Isolation Overhead
 - Quota Enforcement Overhead
-

102. Tenant Acceptance Criteria

Multi-Tenant Architecture는 운영 전에 최소한 다음을 충족해야 한다.

1. 모든 Tenant Resource에 Tenant Context가 적용된다.
2. Tenant ID가 생성 후 변경되지 않는다.
3. API에서 Tenant Isolation이 적용된다.
4. Database Query에서 Tenant Isolation이 적용된다.
5. Search와 Vector Index에서 Tenant Isolation이 적용된다.
6. Memory가 Tenant별로 분리된다.
7. Secret이 Tenant별로 분리된다.
8. Plugin Installation이 Tenant별로 분리된다.
9. Runtime Task가 Tenant Context를 유지한다.
10. Queue Message에 Tenant Context가 존재한다.
11. Cache가 Tenant Scope를 유지한다.
12. Event가 Tenant Scope를 유지한다.
13. Audit와 Usage가 Tenant별로 집계된다.
14. Quota가 Tenant별로 적용된다.
15. Noisy Neighbor 통제가 존재한다.
16. Tenant Suspension이 즉시 적용된다.
17. Tenant Export가 가능하다.

18. Tenant Delete가 Derived Data까지 전파된다.

19. Tenant Migration 절차가 존재한다.

20. Cross-Tenant Security Test를 통과한다.

103. Multi-Tenant Architecture 금지사항

다음 구조는 허용하지 않는다.

- Tenant ID 없는 Tenant Resource
- Client가 전달한 Tenant ID만 무조건 신뢰
- Repository에서 Tenant Filter를 선택 사항으로 구현
- Search 후 Application Layer에서만 Tenant Filter 수행
- Tenant 간 Memory 자동 공유
- Tenant 간 Knowledge 자동 공유
- 다른 Tenant Plugin Credential 사용
- 다른 Tenant Model Credential 사용
- Tenant A의 관리자 Role이 Tenant B에 자동 적용
- Queue Message에서 Tenant Context 제거
- Shared Worker에서 이전 Tenant 상태 유지
- Tenant별 Usage 구분 없는 Model 호출
- Cross-Tenant Cache Key 충돌
- Tenant Secret을 일반 Platform Configuration에 저장
- Data Residency를 무시한 자동 Region 이동
- Tenant Suspend 후 신규 실행 허용
- Tenant 삭제 후 Vector Index에 데이터 잔존
- Platform Admin의 Tenant 데이터 무감사 접근
- Dedicated Tenant와 Shared Tenant에 서로 다른 Domain 의미 적용

- Tenant별 임의 Core Schema 분기
 - 한 Tenant 비용 폭증이 Platform 전체 자원을 소진하는 구조
-

104. Multi-Tenant Architecture 검증 기준

새로운 AGEX Resource나 기능은 다음 질문을 통과해야 한다.

1. 이 Resource는 Tenant Scope인가 Platform Scope인가.
2. Tenant ID는 어디에서 결정되는가.
3. Tenant Context는 전체 호출 경로에서 유지되는가.
4. 다른 Tenant가 Resource ID를 알면 접근할 수 있는가.
5. Database에서 격리가 보장되는가.
6. Cache에서 격리가 보장되는가.
7. Queue와 Event에서 격리가 보장되는가.
8. Search와 Vector Store에서 격리가 보장되는가.
9. Secret과 Credential이 Tenant별로 분리되는가.
10. Runtime Worker가 Tenant 간 상태를 재사용하는가.
11. Quota를 적용할 수 있는가.
12. Usage와 Cost를 Tenant에 귀속할 수 있는가.
13. Tenant Policy를 적용할 수 있는가.
14. Platform Baseline보다 보안을 낮출 수 있는가.
15. Tenant Suspension이 즉시 반영되는가.
16. Region Policy를 지킬 수 있는가.
17. Tenant Export와 Migration이 가능한가.
18. Tenant Delete가 Derived Data까지 처리되는가.
19. Dedicated Deployment로 확장 가능한가.
20. Security Architecture의 Zero Trust 원칙을 충족하는가.

105. Multi-Tenant Architecture의 최종 정의

AGEX Multi-Tenant Architecture는 하나의 AI Operating System에서 다수 Tenant가 독립적인 AI Resource와 실행 환경을 안전하게 운영할 수 있도록 Tenant를 최상위 보안·데이터·정책·비용·운영 경계로 정의하는 Platform Architecture이다.

Tenant는 단순한 고객 식별 값이 아니다.

각 Tenant는 자신의 Agent, Workflow, Knowledge, Memory, Plugin Installation, Model Policy, Secret, Runtime, Usage, Quota 및 Audit Context를 독립적으로 가진다.

모든 API 요청, Service 호출, Execution, Task, Event, Queue Message, Knowledge Retrieval, Memory Retrieval 및 Artifact에는 Tenant Context가 유지되어야 한다.

Tenant Isolation은 API Layer에서만 수행하지 않고 Database, Search, Vector Index, Cache, Queue, Runtime, Secret 및 Observability 계층 전체에서 반복적으로 집행해야 한다.

Shared Infrastructure를 사용하더라도 Tenant 간 데이터와 권한은 공유되지 않으며, 높은 보안·성능·규제 요구가 있는 Tenant는 Dedicated Database, Runtime, Network 및 Encryption Key 구조로 확장할 수 있어야 한다.

Tenant별 Policy와 Configuration은 독립적으로 적용되 Platform Security Baseline보다 낮은 수준으로 설정할 수 없다.

Runtime은 Tenant별 Quota, Concurrency, Cost 및 Fair Scheduling을 적용하여 특정 Tenant의 과도한 사용이 다른 Tenant에 영향을 주지 않도록 해야 한다.

Tenant는 생성, 활성화, 제한, 정지, 종료, Export, Migration 및 삭제의 명확한 Lifecycle을 가지며 Tenant 삭제는 Knowledge, Memory, Artifact, Index, Cache 및 Secret까지 전파되어야 한다.

AGEX Multi-Tenant Architecture는 다음 문장으로 최종 정의한다.

AGEX Multi-Tenant Architecture는 모든 사용자·데이터·AI 자원·실행·정책·비용을 Tenant라는 독립 경계 안에 배치하고, 공유 인프라에서도 그 경계가 절대 혼합되지 않도록 보장하는 AI Operating System의 기본 격리 구조이다.

본 문서는 AGEX Tenant의 정의, Resource 소유권, 데이터·실행·Secret 격리, 자원 공정성, Lifecycle 및 운영 구조를 규정하는 공식 Multi-Tenant Architecture 문서로 사용한다.